A3-PS026-TC560

A Test-Driven Approach to Parser Compatibility and Lossless Identifier Transformation

Team members :

Sania Ruba John Peter , Jemimah Praisy

saniarubajohnpeter@gmail.com

AIORI Hackathon 3

October 13, 2026

Problem Description

Abstract:
AT URIs identify repositories, collections, and records in the AT Protocol. Parsers may interpret the same URI differently. We propose an automated validator that tests syntax, compares two parsers, and checks that two candidate transformations recover the original components exactly.

Problem:
- Different parsers can give different results for the same input
- DID authorities contain colons that complicate parsing
- Transformations may alter or lose components
- Manual testing misses edge cases

Objective: A reproducible tool that finds parser differences and verifies round-trip preservation.

Background

AT URI structure: at://authority/collection/rkey
Authority: repository identifier (e.g., a DID)
Collection: record type
Rkey: individual record

Challenge
- AT URI rules differ from generic RFC 3986 parsing
- DIDs such as did:example:123 contain colons

Scope:
- Syntax validation, parser comparison, and lossless transformation testing.
- Record existence and resolution are out of scope.

Literature & RFC Review

Source	Role in our project
AT URI Scheme Specification	Defines syntax rules, valid and invalid test cases, and the components to preserve
RFC 3986	Generic URI rules used as the comparison baseline for authority and path parsing

Key observations

- The two specifications define syntax at different levels of strictness
- Differences can appear in authority, path, and special-character handling
- Parser behaviour is compared with the spec rather than assumed to be wrong

Proposed Solution

Six-step pipeline

- **Input:** load valid, invalid, and edge-case URIs
- **Validate:** check against AT URI rules
- **Compare:** run Parser A and Parser B
- **Transform:** apply two candidate forward transformations
- **Reverse and verify:** recover and compare components
- **Report:** summarize differences and failures

Key features

- Dual-parser comparison
- Two reversible transformations tested side by side
- Exact component-level verification
- Categorized diagnostic report
- Reproducible, versioned test corpus

Outcome: A modular prototype that detects parser disagreements and transformation failures.

Optimization Proposed

Aspect	Basic approach	Our validator
Parsing	One parser	Two-parser comparison
Coverage	Selected manual examples	Curated automated corpus
Transformation	Inspect output	Forward and reverse test
Preservation	Manual checking	Exact component comparison
Failures	Manual review	Categorized report
Reproducibility	Repeated manual effort	Repeatable test suite

Optimization goals

- Wider coverage of valid, invalid, and edge-case inputs

Proposed metrics

- Parser disagreement rate = disagreeing cases ÷ cases compared × 100
- Round-trip preservation rate = successful round trips ÷ eligible cases × 100

Solution Architecture & Design

Visual: Flowchart or diagram showing the optimization process.

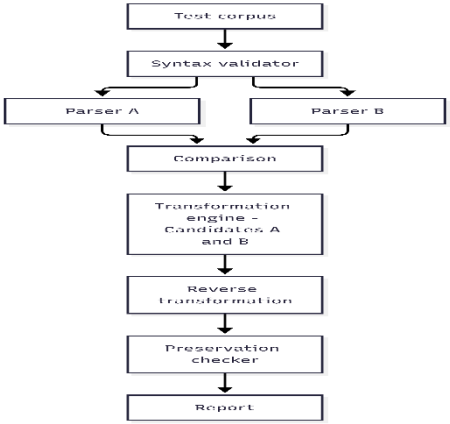


Figure: System Architecture UML Diagram

Data Flow Diagram

Overview of the architecture. Include the flow chart diagram created earlier. Visual: Diagrams or architecture visuals.

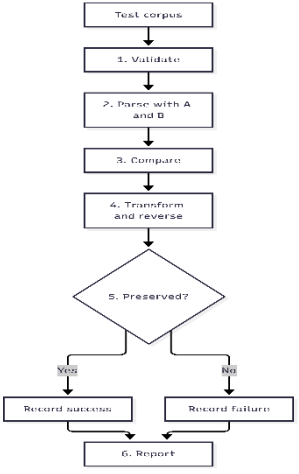


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Day	Milestone
1	Study AT URI rules and RFC 3986
2	Design validation rules and test corpus
3	Build syntax validator and Parser A
4	Build Parser B and candidate transformations
5	Integrate modules and interface
6	Run tests, analyze failures, fix bugs
7	Final integration, documentation, demo

Methodology

- Specification-driven development
- Modular implementation
- Automated unit and regression testing

Deliverable: Working prototype with parser comparisons, transformation tests, and a diagnostic report.

Bibliography I

1. AT Protocol. *AT URI Scheme Specification.* https://atproto.com/specs/at-uri-scheme
2. Berners-Lee, Fielding, Masinter. *RFC 3986: URI Generic Syntax.* IETF, 2005. https://www.rfc-editor.org/rfc/rfc3986.html
3. Python Software Foundation. *urllib.parse.* https://docs.python.org/3/library/urllib.parse.html (if used)
4. [Second parser library: name, version, URL]
5. [Sources for the two transformations]

Thank You for Your Attention!